

The Greedy and Rules-Based Reference Policies in the ARC Disaster-Response Environment

ARC_Game RL — technical reference

June 23, 2026

Abstract

This note documents the two non-learning, API-free reference policies used as baselines: the *greedy* policy (`greedy_decision`) and the *rules-based* policy (`potential_decision`), both in `benchmark_models.py`. They are presented separately: greedy is a purely reactive policy that never invests capital; rules-based reuses greedy’s reactive core and adds a thin capital-investment layer. A final section records their **known limitations** — both are deliberately simple heuristics that under-use the action space and do no forward planning, which is precisely the headroom a learned policy is meant to exploit. Reward terms referenced below are defined in the companion document, *Reward Scoring and Data Provenance*.

Contents

1	The Greedy Policy	1
1.1	Task-choice value function	2
1.2	Worker assignment	2
2	The Rules-Based Policy	2
2.1	State statistics and the demand anchor	2
2.2	Capital investment: build then hire	3
3	Known Limitations (Flaws of these Baselines)	4

How a turn is executed. Both policies return, each round, (i) a set of task-choice selections $\{(\text{taskId}, \text{choiceId})\}$ and (ii) a list of game-action indices. The environment executes *all* of them: `env.step()` takes a comma-separated list of action indices, so a turn may contain arbitrarily many actions of any mix (multiple builds, multiple hires, builds and hires together). Any per-turn limits below are *self-imposed by the policy*, not imposed by the environment.

1 The Greedy Policy

The greedy policy is purely **reactive and myopic**. Each round it does exactly two things:

1. answers open tasks by selecting, for *every* task, the single choice that maximizes an immediate value function (if any choice scores positively); and
2. assigns idle workers (at zero cost) to buildings that are short-staffed.

It **never builds and never hires** — it performs no capital investment whatsoever. It simply spends the starting infrastructure and workforce as efficiently as the current tasks allow.

1.1 Task-choice value function

For an active task t , each available choice c exposes a budget impact $b(c)$ (positive = funding gained, negative = a cost) and a satisfaction impact $s(c)$. With the demand indicator

$$D(t) = \begin{cases} 1 & \text{taskType} \in \{\text{Demand, Emergency}\} \\ 0 & \text{otherwise,} \end{cases}$$

the value of a choice is

$$v(c) = \begin{cases} \frac{b(c)}{10^4} + 10^{-2} s(c), & b(c) > 0 \quad (\text{funding choice}) \\ \mathbf{1}[\alpha(c) \wedge D(t)] + 10^{-2} s(c) - w_c \text{cost}(c), & b(c) \leq 0 \quad (\text{acting / waiting}) \end{cases} \quad (1)$$

where $\text{cost}(c) = -b(c) \geq 0$, $w_c = 0.0002$ (matching the reward’s cost scale), and the *acting* predicate is $\alpha(c) = [\text{cost}(c) > 0] \vee [s(c) \geq 10]$. For each task the policy selects $c^* = \arg \max_c v(c)$, but only records it if $v(c^*) > 0$. The +1 term rewards actually fulfilling a genuine demand/emergency; funding offers are ranked by dollars; per-unit cost is subtracted. This favours the *reliable, immediate* fulfilment options over “free” deferred ones that often fail to complete.

Many tasks, many free answers — in one turn. The value function is evaluated *independently for every open task*, and a selection is emitted for each task whose best choice clears $v > 0$. There is no cap on how many tasks are answered per turn: if several relocation or food tasks are open — including several that can be answered at no cost (a free choice carrying any positive satisfaction signal scores $v = 10^{-2}s > 0$) — the policy answers *all of them* in the same round.

1.2 Worker assignment

The greedy policy also issues zero-cost worker assignments: it groups the enumerated `worker_assignment` actions by target building and, preferring trained workers and larger batches, fills *each* short-staffed building once from the free worker pool.

2 The Rules-Based Policy

The rules-based policy reuses the greedy policy verbatim as its **reactive core**, then *appends* a small amount of capital investment — the one thing greedy never does. The name mirrors potential-based shaping: it spends now against a hand-crafted *state potential* (durable shelter / kitchen / casework capacity and staffed workers) expected to pay off later through the satisfaction and worker-use reward terms.

2.1 State statistics and the demand anchor

From the current facility list it computes

$$\begin{aligned} P &= \sum_{\text{Community } f} \text{currentPopulation}(f) \quad (\text{demand anchor; default 120 if unknown}), \quad (2) \\ \text{shelterCap} &= \sum_{\text{Shelter } f} \text{populationCapacity}(f), \quad n_{\text{kit}} = \#\{\text{Kitchen}\}, \quad n_{\text{cw}} = \#\{\text{CaseworkSite}\}. \end{aligned} \quad (3)$$

Algorithm 1: GREEDYPOLICY (*greedy_decision*)

```
input : game state; enumerated valid actions  $\mathcal{A}$ 
output : choices  $C$ ; action indices  $X$ 
1  $C \leftarrow \emptyset$ ;  $X \leftarrow \emptyset$ 
2 foreach active task  $t$  with choices do
3    $c^* \leftarrow \arg \max_{c \in t} v(c)$  // value function, Eq. (1)
4   if  $v(c^*) > 0$  then
5      $\lfloor$  add  $(t, c^*)$  to  $C$  // every answerable task, no per-turn cap
6  $f_{\text{tr}}, f_{\text{un}} \leftarrow$  free trained, free untrained workers
7 group worker_assignment actions by target building
8 foreach building  $b$  that needs workers do
9   foreach candidate  $a$  of  $b$ , ordered by (trained first, larger quantity) do
10    if  $0 < \text{qty}(a) \leq \text{available workers of } a\text{'s type}$  then
11       $\lfloor$  add index of  $a$  to  $X$ ; decrement that worker pool; break
12 return  $(C, X)$  // note: no construction, no hiring --- ever
```

P is the *current* total displaced population — a **static anchor, not a forecast**. The policy sizes shelter capacity toward P so people can eventually be housed for free in shelters rather than in the recurring-cost motel; it does not anticipate the timing or size of future demand waves (see §3).

2.2 Capital investment: build then hire

After taking greedy's (C, X) , the overlay appends **at most one construction** and **at most one hire**, both gated on horizon and a budget floor:

Build (one building per turn). If $\text{roundsLeft} \geq H_{\min}$ and $\text{budget} \geq B_{\text{res}}$, choose a single target by strict priority and commit it only if it leaves the floor intact:

$$\text{target} = \begin{cases} \text{CASEWORKSITE}, & n_{\text{cw}} < 1 \wedge \text{round} \geq R_{\text{cw}} \\ \text{SHELTER}, & \text{else if } \text{shelterCap} < P \\ \text{KITCHEN}, & \text{else if } n_{\text{kit}} < K_{\text{tgt}} \\ \text{none}, & \text{otherwise,} \end{cases} \quad (4)$$

and the chosen target is committed only if $\text{cost}(\text{target}) \leq \text{budget} - B_{\text{res}}$. The casework site is built first ($R_{\text{cw}} = 0$): a site is staffed only if free workers exist when it is built, so claiming its workers early is the only way it stays operational. *Site selection is geographically blind* — among the available sites for the chosen type it takes the cheapest, but since every construction shares one fixed cost this reduces to the *first-enumerated* available site (see §3).

Hire (one worker per turn). With understaffing $\text{need} = \sum_{f: \text{NeedWorker}} \max(0, \text{requiredWorkforce}(f) - \text{assignedWorkforce}(f))$, a single **hire_untrained** action is appended when $\text{need} > f_{\text{tr}} + f_{\text{un}}$ and $\text{budget} \geq B_{\text{res}}$.

Algorithm 2: RULESBASEDPOLICY (*potential_decision*, baseline mode)

input : game state; round r ; total rounds T
output : choices C ; action indices X

- 1 $(C, X) \leftarrow \text{GREEDYPOLICY}(\text{state})$ // reactive core, Algorithm 1
- 2 compute P , shelterCap, n_{kit} , n_{cw} ; budget, roundsLeft $\leftarrow \max(0, T - r)$
- 3 **if** roundsLeft $\geq H_{\min}$ **and** budget $\geq B_{\text{res}}$ **then**
- 4 target $\leftarrow$ first applicable: CASEWORKSITE ($n_{\text{cw}} < 1 \wedge r \geq R_{\text{cw}}$), SHELTER
 (shelterCap $< P$), KITCHEN ($n_{\text{kit}} < K_{\text{tgt}}$)
- 5 **if** target exists **and** cost(target) $\leq$ budget $- B_{\text{res}}$ **then**
- 6 append one build action (first available site) to X ; budget $\leftarrow$ budget $-$ cost
- 7 **if** need $> f_{\text{tr}} + f_{\text{un}}$ **and** budget $\geq B_{\text{res}}$ **then**
- 8 append one *hire_untrained* action to X
- 9 **return** (C, X)

Constant	Default	Role
$H_{\min}$ (<i>_POT_MIN_HORIZON</i>)	4	Don't build with fewer rounds left — can't amortize.
B_{res} (<i>_POT_BUDGET_-RESERVE</i>)	1500	Floor below which no discretionary build/hire occurs.
R_{cw} (<i>_POT_CASEWORK_-BUILD_ROUND</i>)	0	Earliest round to build the casework site (build-first).
K_{tgt} (<i>_POT_KITCHEN_-TARGET</i>)	2	Operational kitchens to aim for (food + employment).
θ (<i>_POT_SHELTER_-COVERAGE</i>)	∞	Free-shelter re-routing threshold; ∞ disables it.
<i>_POT_MODE</i>	baseline	Selects this policy vs. the <i>demandsupply</i> variant.

Table 1: Rules-based policy constants (*benchmark_models.py*).

3 Known Limitations (Flaws of these Baselines)

Both policies are deliberately simple. The environment permits far richer behavior than either uses; the gaps below are recorded as **flaws**, not features, and are the headroom a learned policy should capture.

1. **Severe under-use of the multi-action turn.** The environment executes a *list* of actions per turn, so a policy may build several buildings, hire several workers, and do both at once. Greedy performs *zero* capital actions; rules-based caps itself at *one* build and *one* hire per turn. Capital therefore ramps at roughly one building per round, which — combined with the ~ 1 -day construction lag — badly bottlenecks early capacity even when budget and multiple free sites are available. A sound policy should build and hire *multiple* units in a single turn when warranted.
2. **No demand forecast / no forward capital investment.** Demand arrives in waves over the episode, but neither policy models this. Greedy is purely reactive; the rules-based anchor P is the *current* population, used only to size shelter capacity. Neither invests *ahead* of an expected

future wave to capture later reward. A sound policy should forecast upcoming demand and pre-build/pre-staff capacity against it.

3. **Geographically blind site selection.** Each candidate site carries a world `position`, communities have positions, and flooding can block relocation routes — yet site selection ignores all of it. Because every construction shares one fixed cost, “cheapest site” degenerates to the first-enumerated available site. The policy never prefers shelters near the communities they serve, never avoids flood-exposed or route-blockable sites, and the `position` field is in fact dropped by the action enumerator before the policy ever sees it.
4. **The budget reserve is dead capital.** $B_{\text{res}} = 1500$ is a hard floor that discretionary build/hire can never cross, so it is hoarded and never deployed for the capacity it ostensibly protects. Worse, reactive task-choice spending is *not* gated by it and can still drive the budget below the floor — so the reserve neither funds infrastructure when needed nor guarantees solvency. A sound policy should be willing to deploy reserves when the expected return justifies it.

Alternate mode. Setting `_POT_MODE=demandsupply` swaps the overlay for a variant that routes each relocation into staffed shelter space via the reliable immediate evacuation while beds last, spilling to the motel only when shelters are full, and sizes shelter capacity to P via a coverage fraction `_POT_DS_COVERAGE`. It shares all four limitations above; the baseline mode is the default reference policy.